ARCHITECTURE REFERENCE

MAGENTRA
The Big Picture

How the engine, the protocol and the desktop app fit together
— the map to come back to before changing anything.

Version	0.16.6
Compiled from	the working tree at commit `71a9d18`
Date	1 August 2026
Source of truth	the code. Where this document and the code disagree, the code is right — §16 lists the drift found while writing it.
Regenerate	`npx electron docs/big-picture/render.mjs`

Contents

1 · What MAGENTRA is

> A headless agentic coding engine, plus a desktop frontend that is only ever a client of it. The engine is UI-agnostic: it consumes typed requests and emits typed events over a versioned protocol, so today's Electron app and tomorrow's VS Code extension are the same kind of thing.

The one idea

Everything below follows from a single rule: **the protocol is the only integration surface.** If the desktop app can do it, it does it by sending a `FrontendRequest` and reading `CoreEvent`s. The app holds no reference into engine internals; the host binary holds none either — it consumes `engine.events` and calls `engine.send()`, and nothing else.

A capability that cannot be expressed as an event/request pair does not exist as far as any frontend is concerned. That is what makes the seam stable enough to build a second frontend against without touching the engine.

What it does

- **Runs an agentic turn loop** against any Anthropic or OpenAI-compatible endpoint — hosted API, gateway, or a model box on the LAN.
- **27 built-in tools**, plus any MCP server's tools, behind a permission engine with two stances.
- **Persists everything** as append-only JSONL transcripts you can resume, rename, archive or delete.
- **Runs up to four workspaces at once**, each in its own engine process, each with its own connection, model, tasks and permissions.
- **Ships as an unsigned desktop app** that updates itself where the OS permits and walks the user through it where it does not.

The design temperament

Three habits show up everywhere in the code and explain most of what looks unusual:

- **Failures that look like success are the enemy.** The finishing rungs (§6), the reuse gate, the stall detector and the runtime evidence floor all exist to catch a turn that *reads* as finished and is not. They remind rather than block — a silent refusal was found to be worse than a loud reminder.
- **One definition, one place.** Token quantities live once in `protocol/tokens.ts`. Provider construction lives once in `providerFactory.ts`. Model-facing prose lives once in the prompt registry. Where a value *must* be duplicated across the app/engine boundary, the comment says so and names its twin.
- **Additive-only state.** A persisted key is added, never renamed or repurposed, so any version can read any other version's state and going back a version is safe. There is no migration machinery, on purpose.

2 · Repository map

```
MAGENTRA/
├── engine/
│   ├── protocol/    types.ts · ndjson.ts · tokens.ts · prompts.ts · branding.ts
│   ├── providers/   Provider interface · anthropic · openai-compat · fake · retry
│   ├── core/        the engine proper — see below
│   ├── tools/       one module per tool + createDefaultRegistry()
│   └── host/        headless binary: NDJSON over stdio
├── app/             Electron desktop frontend (plain JS, outside the TS build)
├── tui/             terminal frontend (ink/TS): a pure NDJSON protocol client,
│                    bundled to resources/engine/tui.mjs and opened by the
│                    `magentra` terminal command
├── docs/            ARCHITECTURE · PROTOCOL · TOOLS · SETTINGS · ADDONS · adr/
├── tools/           version tool (semver/changelog) · prompt-lab
└── benchmarks/      agent benchmark prompts and results
    └── terminal-bench/   Harbor adapter: runs the UNMODIFIED engine on
                          Terminal-Bench 2.0 (driver.mjs is a second NDJSON
                          frontend beside the desktop app; no engine changes)
```

Dependency direction — enforced by TS project references

```
protocol  ←  providers  ←  core  ←  tools  ←  host

protocol   zero dependencies. The seam a future IDE consumes.
core       never imports from tools. Tools are INJECTED via the registry,
           so the engine stays embeddable with any subset (subagents get
           restricted sets).
host       the only package allowed to touch stdin/stdout for user output.
           core has no console.log — everything is an event.
app        plain JavaScript, outside the TS build. Talks to the engine
           ONLY through the host's stdio stream.
tui        TypeScript, inside the TS build (root references). The second
           frontend; speaks the same protocol over the same stdio seam.
```

Inside `engine/core/src`

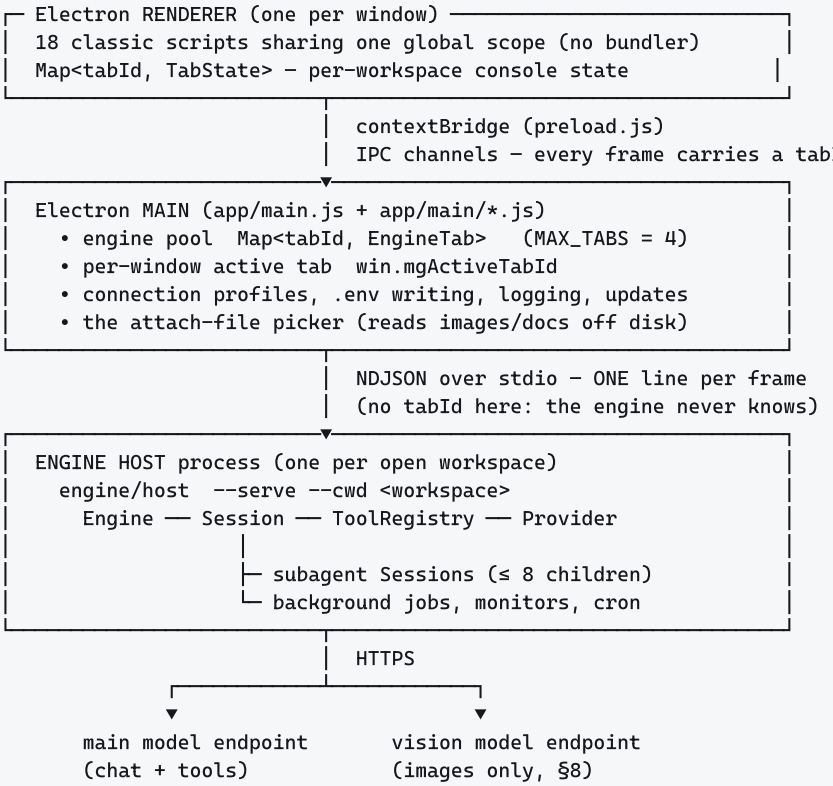
DIRECTORY	WHAT LIVES THERE
`runtime/`	`engine.ts` (request dispatch, slash commands, addons, sessions), `session.ts` (the turn loop — 2 800 lines, the largest file in the repo), `permissions.ts`, `finishing.ts`, `sessionStats.ts`, `fileState.ts`
`agent/`	`tool.ts` (ToolDefinition, registry, ToolContext), `prompts.ts` (system-prompt sections), `addons.ts`, `hooks.ts`, `agents.ts` (subagent types), `builtinAddons.ts`
`config/`	`settings.ts` (the zod schema — single source of truth), `providerFactory.ts`, `pricing.ts`, `frontmatter.ts`
`knowledge/`	`graph.ts` (import graph + PageRank/blast radius), `symbols.ts`, `reuseGate.ts`, `seeds.ts`, `standards.ts`, `workspace.ts`, `docs.ts` (PDF/DOCX text extraction)
`scheduling/`	`background.ts` (background jobs + task-notifications), `cron.ts`, `workflow.ts` (node:vm sandbox)
`state/`	`transcript.ts` (append-only JSONL + repair helpers), `taskStore.ts`
`integrations/`	`mcp.ts` — hand-rolled stdio MCP client (JSON-RPC 2.0, NDJSON framing)
`util/`	`asyncQueue.ts` (single-consumer event stream), `fsAtomic.ts`, `zodToJsonSchema.ts`

Dependencies, and why so few

`@anthropic-ai/sdk`, `zod`, `fast-glob`, `@vscode/ripgrep`, plus `electron` / `electron-builder` / `esbuild` for the app. Everything else — NDJSON framing, diffing, process management, cron parsing, the JSONL store, the MCP client — is hand-rolled because each is small and boring.

3 · Process topology

> Four kinds of process. Three boundaries. Knowing which side of which boundary you are on answers most “why can't I just import that?” questions.



What each boundary costs you

BOUNDARY	CONSEQUENCE
renderer ↔ main	The renderer is sandboxed: it cannot read the transcript file, cannot touch `fs`, cannot spawn. Anything it needs from disk arrives over IPC. This is why `session_restored` ships a flat, render-ready paint list rather than a file path.
main ↔ engine	**The app cannot import from the engine.** The engine ships as a bundled child process. Every value both sides need is duplicated, with a comment naming its twin — `IMAGE_TYPES`, `isLocalBaseUrl`, `VISION_API_KEY_ENV`, `DEFAULT_BASE_URL`. When two such twins disagreed, the app accepted a keyless LAN endpoint the engine then refused to boot on.
engine ↔ provider	The only place a real network call happens. `FakeProvider` substitutes here, which is why the whole suite can run with no network and no key.

Data flow of one user message

```
composer.js
  → window.magentra.sendToEngine({type:"user_message", text, images}, tabId)
  → main.writeToEngine(frame, tabId)        [logs a redacted copy]
  → child.stdin.write(JSON.stringify(frame) + "\n")
  → host: decodeFrames(stdin) → engine.send(frame)
  → Engine.send: case "user_message"
       → withImageDescriptions(text, images)      [§8]
       → startExclusive(...) → Session.runTurn(text)   [§5]
  → CoreEvents ──stdout──→ main (tags with tabId) ──IPC──→ renderer
  → stream.js handleEngineEvent(event) under the owning TabState
```

4 · The protocol — the seam

Version	`PROTOCOL_VERSION = 1`, surfaced to the frontend as `session_started.v`
Defined in	`engine/protocol/src/types.ts` — the source of truth
Framed by	`engine/protocol/src/ndjson.ts`
Documented in	`docs/PROTOCOL.md` (see §16 — five frames are missing from it)

Two transports, one message shape

- **In-process.** `engine.events` is an async-iterable `AsyncQueue`; `engine.send(request)` returns immediately; `engine.idle()` resolves when the running turn ends.
- **NDJSON over stdio.** `encodeFrame(obj) = JSON.stringify(obj) + "\n"`. `decodeFrames` splits on `\n`, tolerates a trailing `\r` (CRLF pipes), skips blanks. A line that fails to parse becomes a non-fatal `error` event rather than killing the transport.

Messages are bare tagged unions discriminated by `type` — no envelope, no correlation id beyond the ones individual frames carry (`turnId`, `permission_request.id`, `question_request.id`, tool-use `id`).

> **The AsyncQueue is single-consumer.**
> Its own header comment: "a second concurrent consumer silently steals events from the first." This is one of the two reasons the engine is hard single-session, and therefore the reason multi-workspace is a process pool rather than a multiplexed session registry (§13).

Core → frontend events

- `session_started` — id, cwd, model, overdrive, the slash-command registry, the addon roster, the per-model rate card
- `turn_started` / `turn_finished`
- `text_delta` / `thinking_delta`
- `tool_call_started` / `tool_call_finished`
- `tool_output_delta` — throttled live tool output
- `retry_status` — provider backing off, and why
- `agent_spawned` / `agent_finished`
- `permission_request` / `question_request`
- `task_list_updated`
- `file_edited` — with a unified diff
- `background_notification` — `start` / `exit`
- `context_update` — the live meters
- `command_output` — slash commands, `!` passthrough, and every finishing-rung marker
- `session_list` / `session_restored` / `session_report`
- `addons_updated` — roster **plus the refreshed command registry** / `addon_draft` / `addon_export`
- `model_catalog` — the endpoint's real `GET /models`
- `cwd_changed` — worktree enter/exit
- `overdrive_changed`
- `error` — with `fatal`

Frontend → core requests

- `user_message` — text + optional `images`
- `steer_message` — mid-turn guidance
- `interrupt`
- `permission_response` / `question_response`

- `slash_command` / `bang_command`
- `set_model` — live, no restart
- `set_connection` — live, no restart (ADR 0007)
- `set_vision` — its own frame, deliberately
- `set_overdrive` / `set_deletion_guard`
- `set_compact_limit`
- `resume_session` / `list_sessions`
- `rename_session` / `archive_session` / `delete_session`
- `stop_background`
- `generate_addon` / `install_addon` / `export_addon`

Two round-trips worth memorising

```
PERMISSION
core → permission_request {id, tool, input, description}
front→ permission_response {id, decision: allow_once|allow_session|
                                    allow_always|deny, message?}
core → tool_call_started → tool_call_finished     (or an error tool_result)

QUESTION  (AskUserQuestion)
core → question_request {id, questions: Question[1..4]}
front→ question_response {id, answers: {"q:0": ["pnpm"]}}
       ↑ keyed POSITIONALLY so two identical question texts cannot collide;
         the exact question text still works as a fallback.
```

> **Why `set_vision` is its own frame.**
>
> `set_connection` rewrites the endpoint, the key and the process environment and rebuilds the provider. That is a great deal of machinery — and a credential rewrite — to move one boolean.

5 · The turn loop

> `Session.runTurn(userText)` — `engine/core/src/runtime/session.ts:1238`. Everything the agent does happens inside this method. Read it once and most of the engine stops being surprising.

Before the loop

1. **`UserPromptSubmit` hook.** A blocking hook ends the turn here with an error event; a non-blocking one contributes context as a reminder.
2. **Plan-first reminder** — fired once while the task list is empty, re-armed when it empties again.
3. **Reset the rung fuses** — `filesChangedThisTurn`, `doubleFilesThisTurn`, `ranCommandThisTurn`, `evidenceNudgeFired`, `incompleteTasksNudgeFired`. They judge *this* turn's work, so they clear together.
4. **Open the stats phase** (root only — children share the ledger, so a subagent must never restart the meter the user is watching).
5. **Emit `turn_started`.**
6. **Clarify pre-layer** — on a genuinely open-ended request, up to three shape-defining multiple-choice questions before any work. Root, attended, `settings.clarify` on; fail-open.
7. **OVERDRIVE snapshot** — `git stash create` parks a dangling commit so an in-workspace deletion stays recoverable. Best-effort; a failure never blocks the turn.
8. **Push the user message** with any pending reminders folded in.

The round

```
for (iteration = 0; ; iteration++) {

  if (capped && iteration >= maxIterationsPerTurn)      break
  if (capped && turnUsage.output > maxTokensPerTurn)    break
        ↑ "capped" is CHILD SESSIONS ONLY. An interactive root turn runs
          uncapped, so the brake is the ladder's per-turn fuses (§6), helped
          by the stall detector below — which only ever REMINDS, and until
          2026-08-22 could not even do that: its signature included each
          result's toolUseId, a fresh provider-random id per call, so no two
          rounds ever matched and it had never once fired.

  drainSteering()          ← queued steer_message lands at this boundary

  { assistant, toolCalls, end } = await streamAssistantTurn(signal)
  addUsage(turnUsage, end.usage)      ← billed cost accumulates
                                        context size does NOT (§9)

  if (verifyBuffered) { … }           ← the silent self-verify reply (§6)

  push(assistant);  stopReason = end.stopReason
  context_overflow → maybeCompact(force), then treated as max_tokens (§9)
  cutoffStreak = max_tokens ? +1 : 0

  if (toolCalls.length === 0)  → THE FINISHING LADDER (§6)

  totalToolCallsThisTurn += toolCalls.length
  results = await executeToolCalls(toolCalls, signal)
      read-class + parallelSafe run CONCURRENTLY
      everything else runs SEQUENTIALLY in call order,
      so permission prompts cannot race

  lastBatchHadError = results.some(isError)
  STALL DETECTOR
      sig = JSON.stringify([calls, results-without-toolUseId])
      3 identical rounds in a row →  force a strategy pivot
      after 2 spent pivots         →  force one concrete question to the user

  push({role:"user", content: withReminders(results)})
  await maybeCompact()                ← mid-turn window squeeze
}
```

After the loop — the `finally` block

- `busy = false`, abort controller cleared, `suppressAssistantText` reset (a turn that died mid-self-verify must not mute the next one).
- **Close the stats phase.** Its total is `T_turn`; its output component is the authoritative `D_final` that supersedes every estimate streamed during the turn. A child never closes the phase.
- Emit `turn_finished` with `usage`, `contextTokens`, optional `contextWarn` and `overdriveSnapshot`. **Cost is deliberately not surfaced** — our counting and a provider's billing can diverge.
- Append a `meta` record snapshotting the tree-wide ledger, so `/resume` restores real accounting rather than a $0.00 session.

Error and interrupt repair

> **A dangling `tool_use` poisons the history.**
> Providers reject a conversation where an assistant `tool_use` has no matching `tool_result` in the next message. So the `catch` synthesises the missing results (`syntheticToolResults` / `unansweredToolUseIds`) *before* recording anything — otherwise an interrupt or a provider error would break not just this turn but every later one, and `/resume` would replay the same wound.

An abort sets `stopReason: "aborted"` ; a real failure classifies the provider error into plain English (`friendlyProviderError`) — a raw `provider returned 401: {json}` means nothing to a user. The original text survives in the desktop layer's engine log.

6 · The finishing ladder

> When the model produces no tool calls, the turn does **not** simply end. It climbs a ladder of checks, each of which can push one more message and `continue` the loop. Order is load-bearing.

1. **Pending steering** — `drainSteering()`
   Outranks every end-of-turn decision. The user's mid-run guidance must be acted on, never dropped by a clean break.

2. **Stop hook** — `end_turn && !stopHookFired && hooks.has("Stop")`
   A blocking hook injects its reason as a system-reminder and continues.

3. **Layer 3 · length cutoff** — `stopReason === "max_tokens" && cutoffStreak ≤ MAX_CUTOFF_STREAK`
   The provider truncated the answer. Resume rather than end on half a sentence. Marker: `↻ continuing after output-length cutoff`. Bounded by its **own** streak counter since 2026-09-01 (it shared `nudgeCount` from 2026-08-22, so an error-recovery nudge spent earlier could end a turn that only needed resuming). The streak counts *consecutive* cutoffs on both emit sites — this one and the tool-call path, which was unbounded — resets on any complete response, and when exhausted the turn ends with a visible `⏸ … cut off N times in a row` instead of a silent `break`. A `context_overflow` stop (input + output filled the window) is compacted first and then handled here as a cutoff; a provider error the classifier reads as overflow (400/413, or an in-band `{"error"}` chunk) compacts and retries the same call, at most twice per turn.

4. **Layer 2 · error recovery** — `end_turn && lastBatchHadError && nudgeCount < MAX_AUTO_NUDGES`
   The last tool batch failed and the turn is ending anyway — weak models bury a failure under a long non-answer. Marker: `↻ auto-recovery…`. **Bounded, and the flag is spent on the nudge** (2026-08-22). This rung was documented as "unbounded on purpose, the stall detector terminates a model that keeps failing identically" — and both halves of that were false. `lastBatchHadError` is only ever assigned after a tool batch, so on a no-tool-call answer it stayed true and the rung re-fired every iteration; and the stall detector neither terminated anything (it only ever `remind`s) nor fired at all (see §5). Because this rung sits above self-verify, a turn whose last batch failed also never self-verified. A later failing batch re-arms the flag, so genuine recovery is still nudged.

5. **Layer 1.5 · incomplete tasks** — `end_turn && !lastBatchHadError && !incompleteTasksNudgeFired`
   Tasks still `pending` / `in_progress`. **Fires once per turn** — without the fuse it re-fired on every end attempt, and a six-task plan charged six extra full-context round trips for one turn's work. It deliberately does not touch `nudgeCount`, which is rung 8's budget.

6. **Runtime evidence floor** — `end_turn && !evidenceNudgeFired && changedCode && (nothingRan || onlyDoubles)`
   One rung, two shapes:
   - **nothing ran** — source was rewritten and nothing was executed, so every claim about it is an inference.
   - **only doubles ran** — something *was* executed, but it was a stand-in this turn wrote, which agrees with whatever the author believed and therefore cannot disprove it.

   Both texts name an honest “I could not run this” as a **complete** answer — because the first shape alone pushes toward the very failure it catches: told “you ran nothing” about a dependency this machine cannot execute, the cheapest way to go quiet is to mock it and watch the mock pass.
   Placed *above* self-verify deliberately: a self-verify that answers DONE breaks the loop where it stands, so anything below it would never run on the turns that go well.

7 **Self-verify** — `end_turn && !selfVerifyFired && totalToolCalls > 0 && overdrive`
Checks the outcome against the original query before the break is allowed. The reply streams **silently** (`suppressAssistantText`); a bare `DONE` ends the turn with no second message, anything else is revealed as genuine follow-up work. **Literal `DONE` only** (2026-08-22): a lone word in any language used to count too, as a safety net for models that localize the sentinel, but that swallowed the one answer that must never end a turn — a model replying `No` to the rung's yes/no question was read as "nothing left to do" and its reply was never rendered. A localizing model now costs one extra round and shows its word.
OVERDRIVE only (since 2026-07-26): an attended turn already has a checkpoint — the user reads the reply — and should not pay a round trip per turn for a second opinion. Skipped entirely for a purely conversational turn: nothing was built, so nothing can be left behind.

8 **Layer 1 · wrap-up** — `≥ 5 tool calls && final text < 150 chars && nudgeCount < 3`
Substantial work ended on a bare reply. Ask for a summary. When files were actually written and a `STANDARDS.md` exists, the nudge also asks the model to confirm the diff complies.

Only after all eight decline does the loop `break` .

Where the prose lives

`engine/core/src/runtime/finishing.ts` holds the rung texts and the pure predicates (`codeFilesAmong` , `runtimeEvidenceText` , `selfVerifyText`). It imports nothing from the session or the permission engine, on purpose, so it can be reasoned about in isolation. Every rung's marker text is a `command_output` event — which is why you can watch the ladder climb in the transcript.

7 · Tools and the permission engine

The tool contract

```
ToolDefinition {
  name, description,
  inputSchema:      zod schema        → validated before execution;
                                        a failure becomes an error tool_result
                                        the model can self-correct from
  permissionClass:  read | mutate | execute | network | interact
  parallelSafe?:    boolean
  isFileEdit?:      boolean
  permissionSubject?(input)           → what a rule matches against
  deletionSubject?(input)             → what the deletion guard sees
  describeInput?(input)               → the human one-liner in the card
  execute(input, ctx, signal)         → ToolResult
}
```

The 27 registered tools

GROUP	TOOLS
Files	`Read` · `Write` · `Edit` · `Glob` · `Grep`
Shell	`Bash`
Tasks	`TaskCreate` · `TaskUpdate` · `TaskList` · `TaskGet` · `TaskStop` · `TaskOutput`
Interaction	`AskUserQuestion`
Delegation	`Agent` · `Workflow`
Network	`WebFetch` · `WebSearch`
Observation	`Monitor` · `GraphQuery`
Git	`EnterWorktree` · `ExitWorktree`
Scheduling	`CronCreate` · `CronDelete` · `CronList` · `ScheduleWakeup` · `PushNotification`
Extension	`Addon`
MCP	any `mcp__<server>__<tool>` wired in at startup

Permission resolution — the real order

```
1. deny rules             settings.permissions.deny
2. protected-path guard   edits into .magentra/** or a .env* file
3. deletion guard         anything that REMOVES a file, folder or worktree
4. allow rules            settings.permissions.allow + allowExact + session grants
5. stance default         ↓
```

STANCE	`READ` / `INTERACT`	`MUTATE`	`EXECUTE` / `NETWORK`
normal	allow	allow	**ask**
OVERDRIVE	allow	allow	allow

File edits are auto-approved in both stances because the review drawer and Undo already cover them. In OVERDRIVE nothing prompts **except** the deletion guard and the protected-path guard — and only a deny rule the user wrote themselves still refuses.

> **"Always allow" stores a shape, not a string.**
> `deriveAlwaysGrant` turns an approved command into its *shape* — `{tool:"Bash", subject:"mkdir", prefix:true}` covers every `mkdir …`; `git push`-style CLIs keep the subcommand; `npm run` keeps the script name; compound or substituted commands stay literal.
> **Shape grants never override the deletion guard**
> , and a protected deletion records no grant at all — it must re-ask every time.

Common behaviours across every tool

- **Freshness.** `FileState` tracks mtime+hash per file read. `Edit` requires a prior read; `Write` requires one for an existing file. Changed on disk since → "re-read first".
- **Truncation.** Default result budget 40 000 bytes (`Read` raises it to 250 000), head+tail with a marker. Several tools also self-cap: Glob at 1 000 paths, Bash at ~30 000 chars.
- **Cancellation.** Every `execute` gets an `AbortSignal`. Bash kills the whole process tree (`taskkill /T /F` on Windows, process-group `SIGKILL` elsewhere).
- **Errors are data.** A failing tool returns `{isError:true}` with an explanation. It never throws into the loop.

8 · The vision path

> The main model is **never** handed a picture — not even one that supports images. A second endpoint looks at it and writes a description; that text is what enters the conversation.

Why it is built this way

- The coding model may sit on an endpoint that does not accept image parts at all.
- The one thing the agent "saw" becomes **auditable text in the transcript**, so every later claim about the picture rests on something checkable.
- A wrapper around the description tells the model outright that it did not see the image and must not claim otherwise.

The chain, end to end

```
~/.magentra/profiles.json
    { name:"fireworks-glm-5p2+vision", model:"…glm-5p2",
      visionProfileId:"546acc4e-…" }        ← points at ANOTHER profile
          │  apply profile in the connection wizard
          ▼
<workspace>/.magentra/settings.json          <workspace>/.env
    vision: true                             MAGENTRA_VISION_API_KEY=…
    visionConnection: { provider, model,      ↑ the key NEVER lands in the
                        baseUrl, profileId }    shareable settings file
          │
          ▼
Engine.send("user_message")
  → withImageDescriptions(text, images)              engine.ts:1014
      guards:  visionUnavailableReason()  ·  ≤ 8 images
               ≤ 9 000 000 base64 chars per image
      emits :  🖼 looking at <name> with <model>…   BEFORE the call,
               because describing runs inside the turn lock and an
               interface that shows nothing reads as a swallowed message
  → Session.describeImageForContext()                session.ts:862
      → describeImage()  →  runInference({images})   session.ts:822
           provider built by createProviderForEndpoint(
             endpointSpecFromSettings(visionConnection, key))
           cached on [provider, baseUrl, contextWindow, apiKey]
           – the key is part of the cache key because it is BAKED INTO
             the provider instance; rotating it otherwise keeps sending
             the old one until the engine restarts
      → OpenAICompatProvider maps the image part to
             { type:"image_url", image_url:{ url:"data:<mime>;base64,…" }}
  → the DESCRIPTION, wrapped, is prepended to the user's text
```

The two gates

```
visionUnavailableReason():
  !settings.visionConnection → "this workspace's connection names no
                                vision model — add one to its profile"
  !settings.vision           → "vision is switched off for this workspace"
  otherwise                  → undefined  (usable)
```

The flag alone is never enough: a switched-on flag with no endpoint behind it can look at nothing. Both the attach picker (which hides image types), the `Read` tool (which refuses `.png`) and the evidence rung ("vision is on" means an image can *actually* be looked at) ask this one function.

Nothing here can fail the turn

An image that cannot be described becomes a plain note saying so, and the typed text still runs. Losing the message because a vision server was down would be the worse outcome.

> **Field note — 1 August 2026.**
> Vision appeared completely dead: settings and profiles were correct, but attached images were dropped with no error and `turn_started` fired 1 ms after `user_message`. Cause: `npm run app` never compiles the engine, and `app/main.js` spawns `engine/host/dist/main.js` — a two-day-old build with no `withImageDescriptions` in it at all.
> **Before debugging any engine behaviour, check `dist/` freshness.**
>
> See §16.

9 · Token algebra and context

> Three quantities, defined exactly once in `engine/protocol/src/tokens.ts` and mirrored for the renderer in `app/renderer/modules/tokens.js`. **They must never be added together.** Nearly every token bug in an agent app is a confusion between two of them.

SYMBOL	NAME	WHAT IT IS
`B(t)`	Current context	The whole **input** of the latest invocation — `inputTokens + cacheWriteTokens + cacheReadTokens`. **Point-in-time.** Does not accumulate: a ten-round turn re-sends a similar prompt ten times, it does not make the window ten times bigger. Generated output is not part of it. *This is what a context meter must show.*
`D(t)`	Deliberation output	Output tokens generated so far by the **current turn**, over every model call it made — auxiliary prompts and subagents included. Starts each turn at 0 and only climbs. The number ticking up in the liveness strip.
`T_turn`	Cumulative turn usage	Every token billed across every invocation of the turn, all four classes. **The cost figure — not the context size.** Arrives in `turn_finished.usage`.

The trap.

`usage.inputTokens` alone *under-reports* the context whenever prompt caching is on, because most of the window arrives as `cacheReadTokens`. That is why `contextTokens` is computed as the full input and shipped as its own field rather than left for the frontend to derive.

Who owns the ledger

`SessionStats` is shared by a session and every subagent it spawns — children hold a reference to the parent's instance. So `/session` and the live meters cover the whole tree, not just the orchestrator. A child **adds to** the open phase; only the root opens and closes one.

Only the **root conversation's** window is measured. A subagent's context never overwrites `B(t)` — otherwise a subagent reading a large file would make the user's own context meter jump.

Compaction

```
maybeCompact() runs
    • mid-turn, after every tool-result batch  (self-gates, so it is cheap)
    • at the end of runTurn
    • on demand via /compact

limit  :  effectiveCompactLimit()
          = floor(contextWindowFor(model, settings) * compactionThreshold)   // 0.8 × the user's window
            lowered by the UI's set_compact_limit cap when one was sent (0 = off)
trigger:  limit > 0  &&  max(stats.contextTokens, estimateContextNow()) >= limit
warn   :  contextTokens >= floor(limit * 0.9)   → contextWarn
recover:  a context_overflow stop, or a 400/413 / in-band {"error"} the provider
          classifies as overflow  → maybeCompact(force) and retry, ≤ 2 per turn

effect :  summarize the OLDEST span through a dedicated call
          (task state, decisions, files touched, open items),
          replace it with one summary message,
          keep the recent tail verbatim.
budget :  summarizerBudget() − reply = 10% of the window (2k..8k, <= maxTokensPerResponse),
          input chunk = window − 2*reply − margin, in chars at CHARS_PER_TOKEN, <= 200k chars;
          per-block clips 1500 (tool call) / 4000 (tool result) chars, head+tail.
          Was a fixed 2000-token reply and a fixed 200k-char chunk (~57k tokens):
          too small a summary on a big window, an overflow of its own on a 32k one.

The JSONL transcript is NEVER rewritten − compaction is a view.
```

The window is the user's number, never a guess.

`contextWindow` is required by the connection wizard and stored once (the settings key); `compactionThreshold` (default 0.8) turns it into the limit. The UI's `set_compact_limit` frame is an optional *cap* on that limit — it can lower it, never raise it — so the TUI, headless runs and subagents, none of which send the frame, compact too. Before this (2026-09-01) the limit's only source was the frame, the UI default was 1,024,000, and a 32k local model ran with auto-compaction effectively off until it died at its wall.

10 · Settings, credentials, state on disk

Four layers, later wins per key

```
1. schema defaults        settingsSchema in config/settings.ts
2. ~/.magentra/settings.json                    (global)
3. <workspace>/.magentra/settings.json          (project)
4. environment variables                         (ENV_OVERRIDES)
```

Unknown keys warn, never crash. Invalid JSON is reported and that file is skipped. `/settings` with no arguments lists every effective leaf value *with the layer it came from*; secrets are redacted there.

The knobs that change behaviour most

KEY	DEFAULT	EFFECT
`provider`	`openai-compatible`	`anthropic` or `openai-compatible`
`model`	`DeepSeek-V4-Flash`	The main model on every turn
`smallModel`	—	WebFetch digestion + compaction summaries; falls back to `model`
`vision` / `visionConnection`	`false` / –	§8
`clarify`	`true`	The open-ended-request question pre-layer
`maxTokensPerResponse`	`32768`	Per-response ceiling; a cutoff triggers rung 3
`maxTokensPerTurn`	`200000`	**Child sessions only** — root turns run uncapped
`maxIterationsPerTurn`	`50`	Same scope
`retention.sessions` / `.tasks`	`100` / `100`	Pruned when a root session starts
`permissions.allow` / `deny` / `allowExact`	`[]`	§7
`hooks`	`{}`	PreToolUse · PostToolUse · UserPromptSubmit · Stop · SessionStart
`mcpServers`	`{}`	Stdio servers; tools land as `mcp__<name>__<tool>`
`reuseCheck.mode`	`remind`	§11 — `"gate"` in an old file loads as `"remind"`
`pricing`	`{}`	Per-model rate card override. No rate card anywhere → token counts with **no** cost estimate, never a guessed price
`worktree.baseRef`	`fresh`	`fresh` = origin's default branch, `head` = current HEAD
`allowInsecureTls`	`false`	Process-wide. The engine warns loudly at boot while it is on

Where credentials live

LOCATION	HOLDS
`~/.magentra/profiles.json`	Named connection profiles **including API keys**, mode `0600`. That is the point of a profile: pick it and you are connected, no re-entry.
`<workspace>/.env`	`MAGENTRA_API_KEY`, `MAGENTRA_VISION_API_KEY`. Written by the app before it sends `set_connection`.
`~/.magentra/settings.json`	`apiKey` when set via `/settings global apiKey …`. Never the project file — that one is shareable.

Resolution is always **env var first, then the stored value**. A keyless local endpoint actively *clears* every key variable the process knows, so "no key configured" and "no key sent" mean the same thing (ADR 0007).

The `.magentra/` directory

PATH	CONTENTS
`settings.json`	Project settings
`sessions/*.jsonl`	Append-only transcripts — one record per line: `message`, `system_prompt`, `permission`, `compaction`, `meta`
`sessions/subagents/`	Child-session transcripts
`sessions/archive/`	Archived transcripts — move a file back to restore it
`tasks/`	Persisted task lists + background-task output files
`worktrees/`	Git worktrees from EnterWorktree — **whole second checkouts**
`addons/`	`name.md` or `name/ADDON.md`
`logs/`	Desktop launch logs, secrets redacted, pruned
`scheduled_tasks.json`	Durable cron jobs
`graph.json` · `symbols.json`	The import graph and symbol index behind GraphQuery
`tmp/`	Engine scratch

`Glob` ignores `.magentra/` **even under** `dot:true` unless the pattern names it explicitly — sweeping up transcripts and second checkouts by accident is a large and silent waste of context.

`STANDARDS.md`

Not a settings key. A file at the workspace root (or `.magentra/STANDARDS.md` — root wins) is injected into the system prompt under "Coding standards (user-provided — binding)". They outrank any default style guidance, and rung 8's wrap-up nudge asks the model to confirm the diff complies. Truncated at 16 KB on a line boundary.

11 · The knowledge layer

> Cheap, deterministic, model-free facts about the workspace. Everything here is a regex scan with an mtime+size cache — there is no AST in this repository, by design.

The import graph — `knowledge/graph.ts`

A directed graph over source files. Node ids are workspace-relative paths with forward slashes; **genuinely external** packages live under synthetic `pkg:<name>` nodes that participate in the graph but are never scanned. A bare specifier naming a sibling package of this workspace (from the root `package.json`'s `workspaces`) resolves to that package's *source* entry instead — its manifest `exports` points at `dist/`, which the scan skips. Persisted to `.magentra/graph.json`, refreshed incrementally.

> **Cache version 3 (2026-08-01).**
> Entries are reused whenever a file's mtime+size are unchanged, so a change to import *extraction* is invisible on any workspace that already has a `graph.json` — every untouched file keeps the edges the old scanner found. `GraphData.version` must therefore be bumped alongside any extraction change; that is what makes the fix reach existing installs. Version 2 fixed two scanner defects: multi-line braced imports produced no edge at all (`RE_FROM`'s `[^;\n]*?` cannot cross a newline), and workspace packages resolved to `pkg:` nodes, which made `slice` / `blastRadius` stop at every package boundary — `types.ts` measured 2 transitive importers against a true 55. Version 3 anchored the side-effect-import pattern to a statement position: `\bimport\s*["']` also matched the word inside a *string*, so `if (path === "/api/import" && req.method === "POST")` invented a package node named `&& req.method ===`.

Pure analytics over it: **PageRank**, **blast radius**, **articulation points**, **dependency slicing**. This is what `GraphQuery` exposes.

TWO TIERS (ADR 0004)

TIER	LANGUAGES	EDGES?
Tier 1	TS/JS, Python, C/C++/Obj-C, Ruby, PHP, Java, Kotlin, Go, Rust	Real edges — resolved directly (`#include "foo.h"`) or by convention (`com.example.Foo` → `**/com/example/Foo.java`)
Tier 2	Everything else, notably Swift and C#	Nodes only. Swift imports modules, not files; a C# `using` names a namespace that need not match any path. Inventing edges there would be guessing.

> **The rule that governs the whole scanner: drop the edge when unsure.**
> A conservative miss is cheap. An invented edge poisons PageRank, the blast radius, and every consumer downstream.

The symbol index — `knowledge/symbols.ts`

Top-level exported names per file. Mirrors `graph.ts` exactly: same caps, same skip rules, same incremental refresh, sibling `.magentra/symbols.json`. `SCAN_EXTS` is shared, so widening one widens the other — and a Go file sent to the TypeScript extractor yields nothing, *silently*.

For Tier 2 languages, symbols are the **only** seed source there is, since those files carry no edges for rank to spread through.

The reuse gate — `knowledge/reuseGate.ts`

When the agent Writes a brand-new source file, this asks — with zero model calls — whether code by a similar name already exists *and* whether the agent did any related search or read this session.

> **It reminds; it never blocks.**
>
> The old refuse-once `"gate"` mode was retired on 2026-07-20. A silent refusal was the root cause of "it suddenly stops": the signal survives a reminder, the refusal doesn't. Fail-open on any uncertainty. The decision table in `evaluateReuseGate` is normative and first-match top-down.

Seeds — `knowledge/seeds.ts`

The one place that decides "which files does this topic concern". Personalised-PageRank seeds come from explicit files, plus query keywords matched against file **paths** and declared **symbol names**. Paths alone are weak — "make the approval card show progress" matches no filename in most repositories — so symbols carry it.

Workspace shape — `knowledge/workspace.ts`

Depth-1, never recursive, no cache: cheap enough to call every turn. It gates how much of a codebase overview the clarify pre-layer can afford before putting a question to the user.

12 · Extension surfaces

The prompt registry — `protocol/prompts.ts`

Every piece of model-facing prose the engine sends is declared once with `definePrompt({id, group, label, channel, …})`, next to the code that uses it, with its default text in the source so the repo stays readable on its own. The registry adds two things: an enumerable catalog for an external editor, and a **per-prompt override read from a plain `.txt` file**, re-read live so editing a file changes the next turn without restarting.

Channels: `system` · `system-conditional` · `reminder` · `tool` · `side-call` · `side-call-user` · `subagent`.

SYSTEM PROMPT ASSEMBLY

```
buildSystemPrompt() =
    behaviorCore()          identity · harness · communication · acting with
                            care · git · code style · task list · working
                            method · autonomy
  + environmentBlock()      cwd · is-git-repo · platform · model · date
  + addonsBlock()           name + description ONLY, one line each
  + extraSections           dynamic sections, plus STANDARDS.md when present

An emptied section header removes the WHOLE section, body and all —
injecting a file's contents with no header would hand the model an
unlabelled wall of text.
```

Addons

A procedure the agent loads on demand: a Markdown file whose frontmatter says *when to reach for it* and whose body says *what to do*. Loaded from built-ins → `~/.magentra/addons/` → `<cwd>/.magentra/addons/`, later tiers replacing earlier ones by name. Layout is either a flat `<name>.md` or a `<name>/ADDON.md` directory whose sibling files are advertised as paths and fetched only if the body calls for them.

Two invariants:

- **Always available.** No enabled state, no toggle, nothing ever injected automatically.
- **Cheap until used.** Only `name` and `description` ride in the system prompt; the body is loaded exactly once, when the `Addon` tool invokes it. The description is therefore a *routing condition*, not a summary — and it is where an expensive addon declares its cost.

WHAT THE GENERATOR TEACHES

An addon exists to buy **predictability** — the same *process* every run, not the same output. `buildAddonPrompt` encodes four rules that carry most of it, so every generated addon inherits them: end each step on a **checkable** condition (an agent that cannot tell done from not-done stops early, and the stopping looks like success); state the target behaviour rather than the ban (*don't think of an elephant* names the elephant); reach for a word the model already holds (*reconnaissance pass*, *blast radius*) instead of three sentences describing it; and cut any line the agent already obeys by default. Full guidance in `docs/ADDONS.md`; the principles are adapted from Mat Pocock's "writing great skills".

> **The frontmatter parser splits at the FIRST colon.**
>
> Both this document's ancestors and the generator prompt once claimed a colon-space inside a value “would start a new key”. It does not — `frontmatter.ts` uses `line.indexOf(":")`, so `description: Use when X happens: then do Y` parses as one key. That false rule was taught to every generated addon until 2026-08-01. The real constraint is one **physical line**
>
> per value; a value that wraps loses its remainder. Pinned by `addon-check.mjs`.

PRECEDENCE — THE USER IS ABOVE THE ADDON

An invoked addon's body is prefixed with one registered header (`addon.invoke-header`, rendered by `addonInvocationHeader`) shared by both entry paths. It says the addon outranks the agent's *default behaviour* — and that the **user outranks the addon**: a message handing a decision back, asking it to stop, or narrowing the request is followed, and the procedure adapts.

> **Why that clause exists.**
>
> The header once said only that an addon's instructions "take priority over general guidance", naming no limit, and it was written out separately at both call sites in two packages. A procedure authored to be relentless — an interview addon — then kept questioning straight through a user delegating the decision back four times and asking it to stop, because nothing anywhere in the system placed the user above a loaded addon. Pinned by `addon-check.mjs`, which also fails if either call site re-inlines its own copy.

NAMING AN ADDON IS INVOKING IT

A leading slash anywhere in a message names an addon: `bana /grill-me yap`, `use /magentron on this`. `addonNamedIn()` detects it on a word boundary (a path like `src/grill-me` does not count, and the longest name wins so `/review` cannot shadow `/review-sql`), the roster block says a named addon is a request to use it, and a reminder points the turn at it.

> **It also suppresses the clarify pre-layer for that turn.**
>
> A message naming an installed addon is not open-ended — the procedure is on disk and its description is already in the prompt. Without the skip, `bana /grill-me yap` was answered with a menu asking *what /grill-me ought to do*: the one question the addon itself already answers. The skip sits outside the `clarify` branch, so naming an addon reaches it whether the pre-layer is on or off.

Users invoke one directly as `/<name>`, dispatched through the same path as the tool. `generate_addon` authors one from plain language: one focused inference call, validated with the real parser, retried with the error appended up to three times. `install_addon` **re-validates** — a draft the user edited is never trusted.

Hooks — `agent/hooks.ts`

User shell commands at `PreToolUse` · `PostToolUse` · `UserPromptSubmit` · `Stop` · `SessionStart`. The JSON payload is piped to stdin; stdout/stderr captured. **Hooks never throw** — a spawn failure yields exit code 127. **Exit code 2 is the block signal**: it stops the action and feeds the hook's stderr back as the reason.

MCP — `integrations/mcp.ts`

Hand-rolled stdio client speaking JSON-RPC 2.0. Per the spec the stdio transport is **newline-delimited JSON** (one message per line, no embedded newlines) — Content-Length framing is *not* used. Handshake stays at 10 s

regardless of config; `tools/call` defaults to 60 s because real MCP tools legitimately run long. A malformed entry is skipped with a warning; a server that fails to start contributes no tools.

Scheduling and background work

PIECE	BEHAVIOUR
`background.ts`	Bash jobs, monitors, background agents. Output streams to `.magentra/tasks/<id>.output`. Completion queues a `<task-notification>` system-reminder for the next turn *and* emits `background_notification` immediately.
`cron.ts`	Fires only when the cron matches the current minute **and** the loop is idle. Session-only unless `durable`. Recurring jobs carry a stable per-job jitter of 0–15 minutes derived from a hash of the job id, to avoid a thundering herd on round times.
`workflow.ts`	Deterministic multi-agent orchestration in a `node:vm` sandbox, with a run journal and resume — an unchanged prefix of `agent()` calls returns cached results instantly.

Subagents

`Session.spawnAgent()`, max 8 live children. A child gets a restricted tool set, its own transcript under `sessions/subagents/`, and **a reference to the parent's `SessionStats`**. `emitFromChild` filters the child's events: turn and text events must not leak into the top-level stream (a frontend waiting for the outer `turn_finished` would otherwise stop on the child's `t_1`), while tool-call events pass through tagged with `agentId` so the UI can still show the work.

13 · Concurrent workspaces

> Up to four workspaces run at once, each fully alive. The decision that made it cheap: **one engine process per tab. The engine core and the wire protocol do not change at all.**

The vocabulary (used verbatim in code)

TERM	MEANING
Workspace	A folder. One `cwd`, one `.magentra/`, one git repo.
Session (= Chat)	One conversation within a workspace, at `.magentra/sessions/<id>.jsonl`. The UI word "chat" and the code word "session" are the same thing.
Engine	One OS subprocess bound to one workspace, hosting one live session.
Tab	A live workspace: its engine + focused session + its slice of renderer state. The unit of concurrency, keyed by a `tabId` the main process mints.

The three rules

1. **Same-folder rule.** At most one live session per folder. Opening a folder that is already open *focuses its existing tab*. This eliminates every shared-`.magentra` and shared-git race **by construction**. Enforced in the main process, so the renderer cannot violate it.
2. **Hard cap of 4.** A fifth open is refused — no eviction, no LRU sleep, no idle timeout. Global across all windows. Memory tops out at four engines.
3. **No cross-tab awareness.** Tabs do not know about each other. No shared engine, no shared state, no messaging.

Why not multiplex sessions inside one engine?

Because the engine is hard single-session in three load-bearing places: a single `this.session` field (replaced, not registered), a **single-consumer** `AsyncQueue`, and one engine-level `busy` / `turnPromise`. Multiplexing would mean a session registry, per-session busy state, a `sessionId` on *every* frame (a protocol break) and a multi-consumer fan-out. And multi-*workspace* would *still* need a pool of those engines — so it buys both complexities and, under the same-folder rule, nothing at all.

Main process — the router

```
engineTabs: Map<tabId, EngineTab>
  EngineTab { id, workspace, model, child, dying, stdoutBuffer,
              stderrBuffer, win }

win.mgActiveTabId          per-WINDOW focused tab
tab.win                    which window owns this tab

outbound:  every engine stdout event is tagged with its tabId and
           routed to tab.win
inbound :  every renderer request names its tabId; an untagged one
           resolves to the SENDER window's active tab

tab.dying  holds a child from a previous stopEngine that has not exited
           yet. A replacement for the SAME tab must never spawn while it
           lives — two engines on one workspace race over its state.
```

Renderer — the state swap

One renderer, not an iframe per tab (that would duplicate the whole UI on top of four engine processes). Instead `Map<tabId, TabState>`, where each `TabState` bundles that tab's console state **including its own detached `streamEl`** — the transcript DOM subtree.

```
applyTabState(tabs[event.tabId]);
handleEngineEvent(event);            ← handlers stay completely unchanged
captureTabState(...);
```

Two rules make it correct:

1. **Enumerate the bundle completely.** Any per-conversation singleton left out leaks across tabs. `TAB_ACCESSORS` is the single source of truth for "what is per-tab": `streamEl`, `busy`, `currentSessionId`, `contextTokens`, `toolRows`, `backgroundJobs`, `permissionQueue`, `activePermission`, `currentTasks`, the live-turn DOM pointers…
2. **Suppress shared-chrome writes for a non-focused tab.** `chromeIsFocused()` guards every chrome updater, so a background tab's turn cannot repaint the focused tab's composer, LED, meter or model picker. On focus change the chrome is re-rendered once from the newly focused state.

Per-tab correctness — the side effects that must stay right

- **Questions and approvals route to the tab that asked.** The card renders into *that* tab's `streamEl`, and the reply goes to *that* tab's engine. In a split view two tabs can have a pending question at once.
- **Each tab may run a different model** and a different connection. The shared composer's model picker edits the **focused** tab. `currentConfig.model` in main is only the *default* a newly opened tab starts on.
- **Connection and vision act on the tab their menu was opened from**, never on whatever is focused — with several workspaces open, "the focused one" is a different workspace with a different connection, and acting on it would silently repoint the wrong engine. The attach picker follows the same rule.

Layout

Multiple open tabs **auto-tile** — the split is simply how multi-tab looks. 2 tabs = two columns; 3 = two on top, one full-width "big" pane below (the user chooses which is big); 4 = a 2×2 grid. Each pane has **its own message input**

wired to that pane's engine, and the shared bottom composer hides. The inspector auto-hides past one tab but stays reopenable.

14 · The desktop app

Main process

FILE	OWNS
`main.js`	Windows, the engine pool, all IPC handlers, the attach-file picker, connection application, workspace opening
`main/config.js`	`config.json` (recents, model, theme, window bounds), atomic JSON writes, settings-file readers, base-URL normalisation, `isLocalBaseUrl`
`main/profiles.js`	`~/.magentra/profiles.json` — CRUD, mode `0600`, sanitising for IPC (the renderer never receives a raw key), vision-pointer resolution
`main/connection.js`	Validating a connection payload, `.env` writing, resolving the vision selection
`main/logging.js`	Per-workspace launch logs, secret redaction, pruning
`main/updates.js`	The two update tiers (§15)
`main/changes.js`	The review drawer and Undo

TWO THINGS MAIN DOES THAT LOOK ODD UNTIL YOU KNOW WHY

- **It reads attachments itself.** Images and documents are read in main, not the renderer (sandboxed) and not the engine. Caps span the whole pending set — 15 files, 2 MB total — because attachments accumulate across repeated "+" clicks. Documents reuse the engine's own dependency-free extractor rather than a second implementation.
- **It writes `.env` and `settings.json` *before* sending `set_connection`.** The engine persists nothing on a swap. A second writer inside the engine would race the first over the same file (ADR 0007).

Preload

`contextBridge` only. The renderer touches nothing else — no `require`, no `fs`, no direct IPC.

Renderer — 18 classic scripts, one global scope, no bundler

```
dom · tokens · state · rain · util · math · markdown · views ·
workbench · tasks · addons · overdrive · tour · stream · setup ·
events · landing · session · updates · tabs · composer
```

Load order in `index.html` **is** the dependency order — there are no imports to express it. `state.js` declares the module-level singletons with `let` rather than `const` specifically so `tabs.js` can reassign them during the per-tab swap.

MODULE	RESPONSIBILITY
`stream.js`	`handleEngineEvent` — the switch every `CoreEvent` lands in
`tabs.js`	`TabState`, `TAB_ACCESSORS`, focus/close, the tiling layout
`composer.js`	Input, attachments, `imageFrameParts`, `composeWithAttachments`, send/steer, the slash palette
`setup.js`	The connection wizard and profile management — including choosing a profile's vision model
`landing.js`	The welcome page and recents
`workbench.js`	Inspector, task rail, meters
`markdown.js` / `math.js`	Hand-rolled rendering — no marked, no KaTeX
`tokens.js`	The renderer's mirror of the token algebra (§9)

ATTACHMENTS: TEXT IS INLINED, IMAGES ARE NOT

```
composeWithAttachments()   text/doc files → inlined into the message with
                           a preamble telling the model they are SNAPSHOTS,
                           not paths on disk (so it does not try to Read them)

imageFrameParts()          images → the frame's `images` array, untouched.
                           The engine replaces them with the vision model's
                           description before the main model ever sees the
                           message.
```

THE SLASH PALETTE IS CARET-ANCHORED, AND THE REGISTRY IS NEVER DERIVED

The palette completes the `/word` under the **caret**, not the start of the input, so `/` offers commands anywhere in a message. The token must begin at a word boundary, which is what keeps `http://host` and `src/foo` from opening it. Completion splices just that token.

Two behaviours split on whether the token is the whole message (`atStart`) — the same test the submit path uses, so the palette and what actually gets dispatched cannot disagree.

WHERE	OFFERS	ENTER
Start of message	every command — built-ins and addons	completes an unfinished name, then submits
Mid-message	**addons only** (`SlashCommandInfo.addon`)	**sends**; `Tab` completes

Mid-message the palette is helping you *name* something, not run it — nothing dispatches from inside a sentence, so completing `/compact` there writes a word that can never fire. An addon's name is the one thing worth completing in prose ("compare this with `/magentron`"). And stealing `Enter` to finish a word nobody asked to finish is the surprise `Tab` exists to avoid.

Two commands are answered by the renderer rather than the engine. `/clear` wipes the local transcript alongside the frame it sends; **`/settings` with no arguments opens the Settings view and sends nothing at all** — the engine's text listing is the right answer for a frontend with no UI and the wrong one here, where the values have a real editor. `/settings <key> <value>` still goes to the engine untouched: that path validates against the schema, writes the file and applies the value live, and no UI replaces it.

> **The command list always comes from the engine.**
> The frontend never builds `/<name>` itself. That registry used to arrive only on `session_started`, so an addon installed mid-session appeared in the Addons view but not under `/` until a restart — the engine would have dispatched it perfectly well. `addons_updated` now carries the same list.

Three themes

`light` · `workbench` · `matrix`. The theme name is mirrored into main because the window's pre-paint `backgroundColor` and the native titlebar overlay are both set before the renderer runs — that mirror is what stops a dark frame flashing ahead of a light UI, the one frame the renderer cannot repaint.

15 · Build, packaging, release, update

Where the engine comes from at runtime

```
engineEntryPoint()                              app/main.js:204

  packaged     process.execPath
               + resources/engine/engine.cjs
               + ELECTRON_RUN_AS_NODE=1
               → one bundled CJS file, no node_modules on disk

  development  node engine/host/dist/main.js
               → the compiled host straight out of the workspace
```

> **The dev launcher does not build.**
> `npm run app` → `app/scripts/launch.js` spawns Electron and nothing else. Main then spawns whatever `dist/` happens to be on disk. Engine source changes are invisible until you run `npm run build` (`tsc -b`) yourself — and the failure is silent, presenting as a feature that simply does not happen. This is a deliberate, known trade-off; it cost a full debugging session on 1 August 2026 (§8).

Packaging — `app/scripts/bundle-engine.js`

1. **The engine** → one CommonJS file via esbuild.
2. **ripgrep** → the right prebuilt `rg` per OS copied in beside the bundle. A missing `rg` for a *target* OS fails the build: shipping an artifact whose Grep is broken must never happen silently.
3. **The app** → main/preload/renderer minified into `build-resources/app`. Dev keeps the readable sources.
4. **doc-extract.mjs** → the engine's document extractor as a standalone ESM bundle, because the attach picker runs in main and reuses it.
5. **tui.mjs** → the terminal UI (`tui/src/cli.tsx`, ink + react + yoga) as one ESM bundle — ESM because ink's reconciler and yoga-layout use top-level await; yoga's wasm rides base64-embedded, so no sidecar ships. `react-devtools-core` is aliased to `app/shims/devtools-shim.cjs`: ink gates devtools behind `process.env['DEV']`, whose bracket access dodges a define fold, and the never-installed optional peer would otherwise become an eager external import that breaks ESM linking at every launch.

The `magentra` terminal command

One name, dispatched by context. An interactive terminal (`-t 0 && -t 1`, no `--gui`) gets the terminal UI — `ELECTRON_RUN_AS_NODE=1 <electron> resources/engine/tui.mjs`, which starts no Chromium, so the sandbox question never arises on that branch. Everything else (desktop entries, double-clicks, `--gui`) gets the GUI exactly as before.

OS	WHERE THE DISPATCH LIVES
Linux	the `afterPack.js` wrapper (the file the deb symlinks to `/usr/bin/magentra` and AppImage execs); sandbox logic below the TUI branch is unchanged
Windows	NSIS setup writes `bin\magentra.cmd` + appends `$INSTDIR\bin` to the HKCU PATH (`app/build/installer.nsh` , removed on uninstall; portable exe has no command). A cmd shim cannot test TTY, so the TUI itself hands no-TTY/ `--gui` launches to the GUI
macOS	`Contents/Resources/bin/magentra` , written by `afterPack.js` (exec bit survives a Windows checkout that way); PATH = one documented symlink

The Windows sandbox rescue

Linux decides the sandbox question *in front of* the process, because Chromium makes that decision before any app code runs and no relaunch can save a packaged build. Windows has no wrapper to put there, so `app/main.js` does it reactively: a renderer that dies **before its first paint** with the sandbox on relaunches once with `--no-sandbox` , logged as `sandbox-rescue-relaunch` . The argv check is the loop guard — a crash *under* `--no-sandbox` is a real crash and stays fatal. It exists because per-user installs under `%LOCALAPPDATA%\\Programs` died at boot on machines whose AppData tree carries reshaped ACLs: every sandboxed child was killed at birth while the same bytes ran fine elsewhere.

What a terminal session establishes before it starts

`magentra` starts wherever the shell happens to be standing, which is the one thing the desktop app never has to reason about — the IDE is always opened *at* something. So a terminal session settles two questions in a fixed order, in `tui/src/trust.ts` and `useEngine` :

1. **Trust.** A folder with no record in `~/.magentra/trusted-folders.json` gets a gate, and until it is answered **nothing is spawned and nothing is written**. Declining quits. The record is global and inherited by subfolders, matched on path *segments* so `…/work` never trusts `…/workspace` ; a marker inside the workspace would be worthless, since the folder being judged could ship one and a clone would arrive pre-trusted. The gate runs **ahead of the profile picker** on purpose: picking a profile writes an API key into `<ws>/.env` , which is itself an act of trusting the folder.
2. **Stance.** A trusted folder's *fresh* session boots into OVERDRIVE — one `set_overdrive` on the first `session_started` . A *resumed* session is left alone: the engine deliberately restores the stance its transcript recorded, and a terminal default must not override a decision the user already made.

The terminal's layout core

Committed transcript lines live inside Ink's `<Static>` , which Ink lays out as an **absolutely positioned, content-sized** box. Two things follow, and both shipped broken: `flexGrow` spacers collapse there, so every right-aligned column printed jammed against its neighbour (`done.out 20 ↑`); and wrapped continuations did not stay under the text they belonged to. `tui/src/markdown.ts` therefore owns wrapping and column arithmetic in **display cells**, and `TranscriptLine` computes right alignment by hand from the terminal width. Because the live streaming line and the committed line now call that same function at the same width, a paragraph reaches its final shape *while* it streams instead of re-wrapping the instant the turn ends. `tui-layout-check.mjs` guards all of it.

Why the terminal repaints on a frame timer

Ink runs on a **legacy React root**, so updates arriving from the host's stdout are not batched: one `setState` per `text_delta` meant one full render, one yoga layout and one terminal repaint *per delta*. Measured against a scripted host on a 2000-delta answer — 10 KB of text the engine wrote in 12 ms — the frontend needed **1587 ms** and emitted **2510 writes / 2.2 MB** of escape output, 216× the text it was showing. That, not the model or the wire, is why the terminal felt slower than the desktop app. Deltas and committed lines now accumulate in refs and flush on a **leading-edge ~30 fps throttle**: the first token still paints immediately, and only a stream already painting inside the last frame is deferred. Same answer: **87 ms, 11 writes, 22 KB**.

Versioning (ADR 0008)

The version is **semver**, three parts. A break is MAJOR, a `feat` is MINOR, **everything else is PATCH** — including `docs`, `chore` and `style`, so every recognised commit moves the version and every push to `main` releases.

The old fourth BUILD part was dropped because `dist.js` truncated it at packaging time, so two releases reported the same version and `semver.eq(latest, current)` answered "no update available" — **the updater was silently dead for every BUILD-only release.** Per-commit traceability moved to the git commit the build carries.

`version.config.json` lists all eight `package.json` targets kept in lockstep (the seven original workspaces plus `tui/`).

Updates — two tiers, decided by install format (ADR 0009)

FORMAT	TIER
Windows NSIS	**self-update**
Linux AppImage / writable	**self-update**
Windows portable	assisted
macOS dmg	assisted
Linux deb, tar.gz	assisted

MAGENTRA ships unsigned and will keep shipping unsigned — that is the constraint the mechanism is built on, not a gap. Squirrel.Mac validates the replacement bundle's signature against the running app, so an unsigned macOS build can *never* replace itself no matter how the updater is written.

Self-update asks first, then downloads with progress and installs on quit; nothing reaches the network before one click. **Assisted** opens the browser on the exact asset for that format — not the releases page — so the user never picks from a list of eight files.

> **The assisted tier is a peer, not a fallback.**
> Roughly a third of installs live there permanently, so it does not sit in a `catch` block. And Windows `portable` must be detected by `PORTABLE_EXECUTABLE_DIR`, never by `process.platform === "win32"` — without that test a portable user is handed the NSIS installer, which silently installs a second copy that then diverges from the one they keep launching.

Testing

The engine is designed to run against `FakeProvider` — scripted turns, no network, no key — over a temp-dir workspace. Today's automated coverage: the version tool (`npm run test:version`) and the app suites (`npm run test:main` — changes, window, connection, reasoning, vision, updates — plus `test:ui` , 32 real Electron scenarios).

`engine/*` still has no unit suite, so its invariants are guarded by purpose-built checks under `.claude/skills/bigboycoding/` , each importing from `dist/` and asserting directly: `permission-check` (23), `addon-check` (28), `tools-check` (61 — every registered tool's contract, plus the read-only ones actually executed), and `glob-state-dir-check` . Write another on the same pattern when you change an engine invariant nothing else guards. `FEATURES.md` tracks per-feature test status honestly, and its rule is worth repeating: *a test that asserts a mock returned what the mock was told to return is not a test.*

16 · Invariants, tripwires and known drift

Invariants — break one and the app breaks in a way that is hard to see

INVARIANT	WHAT BREAKS IF YOU VIOLATE IT
The protocol is the only surface	A private reference from app into engine internals makes a second frontend impossible and the seam untestable.
The event queue has one consumer	A second concurrent consumer silently steals events from the first. This is why multi-workspace is a process pool.
One live session per folder	Two engines writing the same `.magentra/` and the same git repo.
A tab's `dying` child must exit before a replacement spawns	Two engines race over one workspace's state.
Every dangling `tool_use` gets a `tool_result`	The provider rejects the next request, and `/resume` replays the wound forever.
`TAB_ACCESSORS` enumerates every per-conversation singleton	Anything left out leaks across tabs — one workspace's tool rows appearing in another's transcript.
Chrome updaters no-op for a non-focused tab	A background tab's turn repaints the focused tab's composer, LED and meter.
Never add B(t), D(t) and T_turn	A context meter that climbs forever, or a cost figure that is really a window size.
State keys are additive-only	Going back one version stops being safe, and MAGENTRA has no migration machinery on purpose.
Drop the graph edge when unsure	An invented edge poisons PageRank, blast radius and every consumer downstream.
The finishing ladder's order	Anything ranked below self-verify never runs on a turn that goes well, because a DONE breaks the loop where it stands.
Every rung is bounded per turn	Every re-fire is a full-context round trip. The incomplete-tasks rung once charged six of them for one six-task plan. Most rungs use a one-shot boolean fuse; layers 2 and 3 share the `nudgeCount` budget (`MAX_AUTO_NUDGES`). Until 2026-08-22 those two checked no bound at all — they incremented `nudgeCount` without ever reading it, which on an uncapped root turn is a non-terminating loop.

Mirrored constants — change both or ship a silent contradiction

VALUE	ENGINE SIDE	APP SIDE
Image types	`tools/src/read.ts`	`main.js IMAGE_TYPES`
Local/LAN endpoint test	`config/providerFactory.ts`	`main/config.js isLocalBaseUrl`
Vision key env var	`config/settings.ts`	`main/config.js VISION_API_KEY_ENV`
Default base URL	`config/settings.ts`	`main/config.js DEFAULT_BASE_URL`
Theme names + order	—	`main/config.js ⇔ renderer/state.js`
Token algebra	`protocol/tokens.ts`	`renderer/modules/tokens.js`

When `isLocalBaseUrl` and its twin disagreed, the app accepted a keyless LAN endpoint that the engine then refused to boot on. `app/tests/connection.test.js` asserts they agree.

Known drift — documentation behind the code, found while writing this

WHERE	DRIFT
`docs/SETTINGS.md`	~~Documents `compactionThreshold: 0.8`, which was not in `settingsSchema`~~: Resolved 2026-09-01: the key exists and is the mechanism (§9).
`docs/PROTOCOL.md`	Five frames exist in `types.ts` but are undocumented: `set_model`, `set_compact_limit`, `steer_message`, `session_report`, `overdrive_changed`. `set_connection` is named but has no section.
`docs/SETTINGS.md`	The “Embeddings” section is an empty table with no keys.
`docs/ARCHITECTURE.md`	Says per-project instruction files “are not read yet”, but `STANDARDS.md` is loaded into the system prompt (`knowledge/standards.ts`). It also says no engine test framework is wired up, while `app/tests/` now carries six suites.

None of these change behaviour — they change what a reader believes the behaviour is, which is the more expensive kind of error.

17 · Concept → file index

IF YOU ARE LOOKING FOR…	GO TO
The turn loop, every rung, compaction, subagent spawn	`engine/core/src/runtime/session.ts`
Request dispatch, slash commands, addon generation, session list/resume/rename/archive, live model & connection swap	`engine/core/src/runtime/engine.ts`
Every event and request shape	`engine/protocol/src/types.ts`
Token definitions	`engine/protocol/src/tokens.ts`
Prompt registry, override files, channels	`engine/protocol/src/prompts.ts`
System-prompt sections	`engine/core/src/agent/prompts.ts`
Finishing-rung prose and predicates	`engine/core/src/runtime/finishing.ts`
Permission resolution, deletion guard, grant shapes	`engine/core/src/runtime/permissions.ts`
The settings schema — the source of truth	`engine/core/src/config/settings.ts`
Turning settings into a Provider	`engine/core/src/config/providerFactory.ts`
Image parts on the wire	`engine/providers/src/openai-compat.ts`
Tool registration list	`engine/tools/src/index.ts`
NDJSON server, shutdown, interrupt-on-EOF	`engine/host/src/serve.ts`
Engine pool, tab routing, IPC, attachments	`app/main.js`
Profiles and the vision pointer	`app/main/profiles.js`
Connection validation and `.env` writing	`app/main/connection.js`
`TabState`, the swap, tiling	`app/renderer/modules/tabs.js`
The event switch	`app/renderer/modules/stream.js`
Attachments and the send path	`app/renderer/modules/composer.js`
The connection wizard	`app/renderer/modules/setup.js`
Engine bundling and ripgrep	`app/scripts/bundle-engine.js`
Why a decision was made	`docs/adr/` · `CONTEXT.md`

Honest test status	`FEATURES.md`

The five files that explain the most per line

1. `engine/protocol/src/types.ts` — 449 lines. The whole contract.
2. `engine/core/src/runtime/session.ts:1238–1735` — `runTurn`. The agent's entire behaviour.
3. `engine/core/src/config/settings.ts` — every knob, each with a comment saying *why* it exists.
4. `docs/CONCURRENT-WORKSPACES.md` — the multi-tab design and the rejected alternative.
5. `CONTEXT.md` — the ubiquitous language. Short, and it names the concepts the code assumes you already hold.

MAGENTRA 0.16.6 · compiled 1 August 2026 from the working tree at `5da5d85`.
Source: `docs/big-picture/big-picture.html` · rebuild with `npx electron docs/big-picture/render.mjs`.
Where this document and the code disagree, the code is right.